

Auditoría de Seguridad con IA: OWASP · Vulnerabilidades · Pipeline Seguro

Ejercicio de Clase · Ejercicio 3 de 3 · Auditoría OWASP + Corrección + Pipeline de Seguridad

Nombre del alumno/a	Angel Echenique
Fecha	05/04/2026
☐ Enlace GitHub del repositorio	

☐ **Instrucciones:** Ejercicio de auditoría de seguridad: analizarás código con vulnerabilidades reales, construirás el prompt de auditoría siguiendo OWASP Top 10, propondrás las correcciones y diseñarás un pipeline CI/CD con etapa de seguridad integrada.

BLOQUE 1 — Código vulnerable para auditar

El siguiente código Python (API con FastAPI) contiene múltiples vulnerabilidades de seguridad. Lee el código completo antes de responder:

```
# api_usuarios.py — CÓDIGO VULNERABLE (solo para ejercicio)

from fastapi import FastAPI
import sqlite3, os

app = FastAPI()

DB = 'usuarios.db'
SECRET = 'clave123' # ← hardcoded

def get_conn():
    return sqlite3.connect(DB)

# ENDPOINT 1: Login
@app.post('/login')
def login(email: str, password: str):
    conn = get_conn()
    cursor = conn.cursor()
    query = f"SELECT * FROM users WHERE email='{email}' AND password='{password}'"
    cursor.execute(query)
    user = cursor.fetchone()
    if user:
        return {'status': 'ok', 'user': user} # devuelve fila completa con hash
    return {'status': 'error'}

# ENDPOINT 2: Registro
@app.post('/registro')
def registro(nombre: str, email: str, password: str, edad: int):
    conn = get_conn()
    conn.execute(
```

```
f"INSERT INTO users VALUES ('{nombre}', '{email}', '{password}', {edad})"
) # ← sin validaciones de edad ni formato
conn.commit()
return {'ok': True}

# ENDPOINT 3: Obtener usuario (sin autenticación)
@app.get('/usuario/{id}')
def get_user(id: int):
    conn = get_conn()
    row = conn.execute(f'SELECT * FROM users WHERE id={id}').fetchone()
    return row # expone todos los campos incluyendo password

# ENDPOINT 4: Eliminar (sin verificar permisos)
@app.delete('/usuario/{id}')
def delete_user(id: int):
    conn = get_conn()
    conn.execute(f'DELETE FROM users WHERE id={id}')
    conn.commit()
    return {'deleted': id}
```

1.1 — Primera lectura: identifica vulnerabilidades preliminares

Sin usar la IA todavía, señala las vulnerabilidades que detectas a simple vista. Para cada una indica el tipo OWASP:

#	Línea / Endpoint	Vulnerabilidad detectada	Categoría OWASP
1	7	Secretos Hardcodeados	A07:2021. Exposición de credenciales críticas en el código fuente.
2	16	SQL Injection	A03:2021. Permite bypass de autenticación y robo de base de datos.
3	19	Exposición de datos sensibles	A02:2021. Se envía el hash de la contraseña al cliente en la respuesta JSON.
4	26	Command Injection	A03:2021. Permite ejecutar comandos arbitrarios en el servidor a través de os.system.

BLOQUE 2 — Construye el prompt de auditoría de seguridad

- Recuerda el estándar OWASP Top 10: las vulnerabilidades web más críticas. Un prompt de auditoría profesional debe indicar el estándar, el contexto, el tipo de análisis y el nivel de detalle esperado (visto en la sección 2.7 del módulo).

Escribe el prompt completo que enviarías a la IA para realizar una auditoría OWASP Top 10 de este código. Recuerda incluir los 6 elementos del prompt de auditoría vistos en clase:

Los 6 elementos del prompt de auditoría

- **Rol Experto:** Definir que la IA actúe como un especialista en Ciberseguridad.
- **Contexto/Código:** Indicar que se trata de una API con FastAPI y SQLite.
- **Estándar de Referencia:** Solicitar el análisis basado específicamente en **OWASP Top 10**.
- **Entorno Técnico:** Especificar que trabajamos con **Python 3.10**.
- **Tipo de Ayuda:** Pedir análisis de causa raíz, explicación de riesgos y código corregido.
- **Justificación:** Exigir que explique *por qué* cada solución es necesaria para mitigar el riesgo.

Aplicando los 6 elementos en un prompt

Actúa como un experto en auditoría de ciberseguridad.

Analiza el siguiente código de una API desarrollada en **FastAPI** y **SQLite** bajo el estándar **OWASP Top 10**.

Detecta todas las vulnerabilidades presentes, explica cada riesgo detalladamente e indica su **causa raíz**. Proporcióname el código corregido aplicando parches de seguridad y justifica técnicamente por qué estas soluciones son necesarias para proteger el sistema en un entorno de producción con **Python 3.10**.

Aquí tienes el código: [PEGAMOS EL CÓDIGO DE BLOQUE 1]

2.2 — Tabla de auditoría OWASP

Completa la tabla de auditoría para cada vulnerabilidad encontrada en el código. Usa la respuesta de la IA y tu propio análisis:

Vulnerabilidad	OWASP Ref.	Línea / Endpoint	Riesgo concreto	Solución propuesta
SQL Injection	A03:2021	login(), registro(), get_user()	Un atacante puede bypassar el login usando ' OR '1'='1 o borrar toda la base de datos inyectando comandos maliciosos en los strings.	Usar consultas parametrizadas (placeholders ?) que separan los datos de la instrucción SQL, impidiendo que el input se ejecute como código.
Exposición de datos	A02:2021	/login y /usuario/{id}	Se envía el hash de la contraseña y datos internos al cliente. Si un atacante intercepta esto, puede intentar romper el hash mediante fuerza bruta.	Implementar modelos de respuesta (Schemas) con Pydantic que filtren solo los campos públicos (email, nombre) y excluyan la contraseña.
Credenciales hardcoded	A02:2021	SECRET = 'clave123'	Si el código se sube a un repositorio público (como GitHub), cualquiera tendrá la llave maestra para firmar tokens o cifrar datos del sistema.	Mover la clave a un archivo .env o a un Gestor de Secretos y cargarla mediante variables de entorno en el sistema.
Sin autenticación	A01:2021	/usuario/{id} (GET/DELETE)	Permite el "Broken Access Control": cualquier persona con acceso a internet puede consultar o eliminar cualquier usuario solo cambiando el ID en la URL.	Implementar autenticación JWT y una lógica de verificación que compruebe si el usuario que solicita la acción es el dueño de la cuenta o un administrador.
Sin validaciones	A03:2021	registro() — edad, email	Permite introducir correos falsos o edades imposibles (ej. -50), lo que puede causar errores lógicos en el backend o ataques de inyección de datos.	Usar tipos de datos específicos como EmailStr y restricciones de rango (ej. ge=18) en los modelos de entrada de FastAPI.
Información excesiva	A05:2021	return row completo	Revela la estructura interna de la base de datos y campos que el frontend no necesita, facilitando el reconocimiento para un futuro ataque.	Definir una interfaz de salida mínima que devuelva exclusivamente lo necesario para la funcionalidad solicitada.

BLOQUE 3 — Diseña el endpoint seguro

Elige UNO de los endpoints vulnerables (/login o /usuario/{id}) y describe cómo quedaría la versión segura. Especifica todos los cambios de seguridad aplicados:

✗Código vulnerable (aspectos)	✓Versión segura (mejoras aplicadas)
Consulta SQL con f-string directa	Consultas parametrizadas: Se usan "placeholders" (?) para que el motor de la base de datos trate el input como texto plano y nunca como código ejecutable.
Sin verificación de tipos de entrada	Validación con Pydantic: Se definen clases (Schemas) que obligan a que el email tenga formato de correo y la contraseña cumpla con una longitud mínima antes de procesar nada.
Devuelve todos los campos del usuario	Modelos de Respuesta: Se utiliza un filtro (response_model) para que la API solo devuelva el ID, nombre y email, eliminando el hash de la contraseña de la respuesta.
Sin autenticación / token JWT	Implementación de JWT: En lugar de devolver un simple "ok", el sistema genera un token cifrado y temporal (JSON Web Token) que el usuario usará para identificarse en adelante.
Contraseña en texto plano	Hashing de contraseñas: Las contraseñas nunca se comparan directamente. Se usa una librería (como passlib con bcrypt) para verificar si el hash de lo que escribió el usuario coincide con el guardado.

```
@app.post('/login', response_model=UserPublic) # Filtra la salida automáticamente
def login(datos: UserLogin): # 'datos' ya viene validado por Pydantic
    conn = get_conn()
    cursor = conn.cursor()

    # 1. Consulta parametrizada (Seguridad contra SQL Injection)
    query = "SELECT * FROM users WHERE email = ?"
    cursor.execute(query, (datos.email,))
    user = cursor.fetchone()

    # 2. Verificación de Hash (Nunca comparamos texto plano)
    if user and verify_password(datos.password, user['password_hash']):
        # 3. Generación de JWT (Sesión segura)
        token = create_access_token(data={"sub": user['email']})
        return {"access_token": token, "token_type": "bearer"}

    raise HTTPException(status_code=401, detail="Credenciales inválidas")
```

Explicación de los cambios

- **El fin de la Inyección SQL:** Al usar `?` en la consulta, si un atacante intenta escribir `' OR '1'='1'`, la base de datos buscará literalmente a un usuario cuyo email sea exactamente ese texto extraño, en lugar de ejecutar la lógica maliciosa.
- **Principio de Menor Privilegio:** Al usar un `response_model`, protegemos la privacidad. El frontend "no necesita saber" cómo luce el hash de la contraseña del usuario; solo necesita su nombre y email para la interfaz.
- **No confíes en el usuario (Zero Trust):** Validar los tipos de entrada con Pydantic evita que nos envíen archivos maliciosos o textos gigantescos en campos donde esperamos un simple entero o un correo corto.
- **Almacenamiento Seguro:** En el código original, si alguien robaba la base de datos, tenía todas las contraseñas. Con el hashing (bcrypt), aunque roben la base de datos, los atacantes solo verán cadenas de texto aleatorias imposibles de revertir.

BLOQUE 4 — Pipeline de Seguridad CI/CD (conexión con Módulo anterior)

- ❑ Conexión con el módulo de Pipeline: un pipeline profesional no solo ejecuta tests, también incluye una etapa de análisis de seguridad estático (SAST) que detecta vulnerabilidades antes de llegar a producción.

4.1 — Diseña el workflow de seguridad

Completa el diseño del fichero GitHub Actions que integra análisis de seguridad. Para cada job indica su nombre, cuándo se ejecuta y qué herramienta usa:

Job / Etapa	Cuándo ejecutar	Herramienta sugerida	¿Qué detecta / valida?
lint	En cada push	flake8 / pylint	Errores de sintaxis y cumplimiento de estándares de estilo (PEP8).
test	En cada push	pytest	Que la lógica de la API (login, registros, etc.) funcione como se espera.
coverage	En cada push	pytest-cov	El porcentaje de código que realmente está siendo probado por los tests.
security-sast	En PR a main	Bandit / semgrep	Vulnerabilidades estáticas como inyecciones SQL o funciones peligrosas (<code>os.system</code>).
dependency-check	En PR a main	safety / pip-audit	Librerías instaladas que tienen vulnerabilidades conocidas (CVEs).
secret-scan	En cada push	truffleHog / detect-secrets	Claves, tokens o contraseñas (como nuestro SECRET) subidas por error.
deploy	Push a main (si todo pasa)	GitHub Actions Deploy	Despliegue automático a producción solo si el código es funcional y seguro.



4.2 — Escribe el paso security-sast en YAML

Escribe el bloque YAML del job 'security-sast' usando Bandit para Python. Incluye: nombre, trigger, steps de checkout, setup-python, instalación de bandit y ejecución con output:

```
Y name: Security Audit ADEV Untitled-1
1  name: Security Audit ADEV
2
3  on:
4    pull_request:
5      [cite_start]branches: [ "main" ] # Se activa al intentar fusionar código a la rama principal [cite: 33]
6
7  jobs:
8    security-sast:
9      runs-on: ubuntu-latest
10     steps:
11       - name: Checkout code
12         [cite_start]uses: actions/checkout@v4 # Clona el código para analizarlo [cite: 33]
13
14       - name: Set up Python
15         uses: actions/setup-python@v5
16         with:
17           [cite_start]python-version: '3.10' # Asegura compatibilidad con nuestro entorno [cite: 33]
18
19       - name: Install Bandit
20         [cite_start]run: pip install bandit # Instala el auditor de seguridad [cite: 33]
21
22       - name: Run Bandit Security Scan
23         # -r: escanea carpetas recursivamente
24         # -f txt: genera un reporte legible
25         # -o: guarda el resultado en un archivo
26         run: bandit -r . [cite_start]-f txt -o security_report.txt [cite: 33]
27
28       - name: Upload Security Report
29         uses: actions/upload-artifact@v4
30         with:
31           name: bandit-report
32           path: security_report.txt # Permite descargar el reporte si el pipeline falla
33
```

BLOQUE 5 — Investigación y síntesis final

□ Este bloque requiere que investigues por tu cuenta. Usa una fuente fiable y cita siempre de dónde lo sacaste.

5.1 — Investiga una herramienta de seguridad para IA/código

Investiga una herramienta que use IA para análisis de seguridad en código (ej: GitHub Advanced Security, Snyk, Semgrep, SonarCloud...):

Campo	Tu respuesta
Nombre de la herramienta	Snyk (DeepCode AI).
¿Cómo detecta vulnerabilidades con IA?	Usa IA Híbrida para realizar análisis semántico y de flujo de datos, entendiendo la intención del código y no solo patrones de texto.
¿Se integra en GitHub Actions?	Sí, mediante una Action oficial que permite detener despliegues si el nivel de riesgo es alto.
¿Detecta OWASP Top 10?	Sí, cubre las 10 categorías, incluyendo Control de Acceso y Fallos Criptográficos.
Ventaja principal vs auditoría manual	Capacidad de análisis a escala: puede revisar miles de líneas de código y sus dependencias en segundos, algo imposible para un humano.
Fuente consultada (URL)	snyk.io / snyk.io/platform/deepcode-ai/
Fecha de consulta	05/04/2026

1. ¿Cómo detecta Snyk las vulnerabilidades con IA?

Snyk utiliza un motor llamado **DeepCode AI**. A diferencia de otras IAs que solo "predicen" texto, esta es una **IA Híbrida** (combina IA simbólica con aprendizaje profundo) entrenada con miles de millones de líneas de código seguro.

Ejemplo Práctico en tu código de la API:

Imagina que tienes este código (visto en el Bloque 1):

```
query = f"SELECT * FROM users WHERE id={id}"
```

- **Un escáner básico:** Diría "Veo una f-string en un SQL, sospecho que es peligroso".
- **Snyk con IA:** Realiza un **Análisis de Flujo de Datos (Taint Analysis)**. Rastrea la variable id desde que entra en el endpoint /usuario/{id} hasta que llega a la base de datos.
- **La conclusión de la IA:** "He detectado que el dato que viene del usuario (id) llega a una consulta crítica sin ser validado ni transformado en entero. Esto es un **SQL Injection (A03:2021)**. En lugar de solo avisarte, te sugiero usar un modelo de Pydantic o parámetros en la consulta".

2. ¿Cómo se integra en GitHub?

Snyk es famoso por su integración casi invisible pero potente. Se puede hacer de dos formas principales:

A. Integración Directa (GitHub App)

Es la más sencilla. Conectas Snyk a tu cuenta de GitHub y:

- **Escaneo de Repositorios:** Analiza tu código cada vez que haces un git push.
- **Pull Request Checks:** Si intentas subir código vulnerable, Snyk pone un "X" roja en el PR y añade un comentario explicando el fallo y cómo arreglarlo.
- **PR de Corrección Automática:** ¡Lo mejor! Snyk puede abrirte automáticamente un Pull Request que corrige la vulnerabilidad (por ejemplo, actualizando una librería vieja a una segura).

B. Integración en el Pipeline (GitHub Actions)

Como vimos en el Bloque 4, puedes añadirlo como un paso de tu workflow. Aquí un ejemplo de cómo se vería ese paso en mi archivo.

```
- name: Snyk Open Source Analysis
  uses: snyk/actions/python@master
  env:
    SNYK_TOKEN: ${ secrets.SNYK_TOKEN } # Usa un secreto para la API Key
  with:
    command: monitor # Registra el proyecto en el panel de Snyk para vigilancia 24/7
```



5.2 — Reflexión final integradora

Conecta los tres módulos aprendidos: has depurado errores con IA, refactorizado con modularización y ahora añadido seguridad al pipeline. ¿Cómo cambia la calidad y seguridad del software cuando se integran estos tres elementos de forma sistemática? ¿Qué ocurre si alguno de ellos falta? Razona con ejemplos del código de este ejercicio.

La integración de la **depuración con IA**, la **modularización** y la **seguridad automatizada** transforma el desarrollo de software de una tarea artesanal y reactiva en un proceso de ingeniería **proactivo y resiliente**.

Esta unión crea una sinergia donde cada elemento protege al otro:

- **La Depuración con IA** garantiza que el código sea funcional y libre de errores lógicos desde su nacimiento, actuando como un mentor que eleva nuestro criterio técnico.
- **La Modularización** organiza esa lógica en compartimentos estancos, permitiendo que el software sea mantenible y escalable; es decir, que lo que construimos hoy no sea una pesadilla mañana.
- **La Seguridad en el Pipeline (Snyk)** actúa como la red de seguridad final. Sin ella, podríamos tener un código "limpio" y "modular" que, irónicamente, es una invitación abierta al desastre.

En conclusión: Desarrollar sin integrar estos tres pilares es como construir una casa con paredes hermosas y una arquitectura envidiable (modularización), pero con los cimientos agrietados (errores de lógica) y la puerta principal abierta de par en par (vulnerabilidades).

Al unirlos, dejamos de "apagar incendios" para convertirnos en creadores de sistemas robustos, donde herramientas como Snyk y la IA actúan como un guardia de seguridad y un consultor experto que nunca duerme, garantizando que el software no solo funcione, sino que sea **intrínsecamente seguro**.